



第3章 快速傅里叶变换

3.1 引言

3.2 直接计算DFT的问题及改进的途径

3.3 按时间抽取（DIT）的基2-FFT算法

3.4 利用FFT分析时域连续信号频谱

3.5 FFT的其他应用

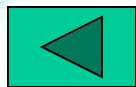




3.1 引言

快速傅里叶变换（FFT）并不是一种新的变换，而是离散傅里叶变换（DFT）的一种快速算法。

由于有限长序列在其频域也可离散化为有限长序列（DFT），因此离散傅里叶变换（DFT）在数字信号处理中是非常有用的。例如，在信号的频谱分析、系统的分析、设计和实现中都会用到DFT的计算。但是，在相当长的时间里，由于DFT的计算量太大，即使采用计算机也很难对问题进行实时处理，所以并没有得到真正的运用。直到1965年首次发现了DFT运算的一种快速算法以后，情况才发生了根本的变化。人们开始认识到DFT运算的一些内在规律，从而很快地发展和完善了一套高速有效的运算方法，这就是现在人们普遍称之为快速傅里叶变换（FFT）的算法。FFT出现后使DFT的运算大大简化，运算时间一般可缩短一二个数量级之多，从而使DFT的运算在实际中真正得到了广泛的应用。





3.2 直接计算DFT的问题及改进的途径

3.2.1 直接计算DFT的运算量问题

设 $x(n)$ 为 N 点有限长序列，其DFT为

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{nk} \quad k=0, 1, \dots, N-1 \quad (3-1)$$

反变换（IDFT）为

$$X(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-nk} \quad n=0, 1, \dots, N-1 \quad (3-2)$$





二者的差别只在于 W_N 的指数符号不同，以及差一个常数乘因子 $1/N$ ，所以IDFT与DFT具有相同的运算工作量。下面我们只讨论DFT的运算量。

一般来说， $x(n)$ 和 W_N^{nk} 都是复数， $X(k)$ 也是复数，因此每计算一个 $X(k)$ 值，需要 N 次复数乘法和 $N-1$ 次复数加法。而 $X(k)$ 一共有 N 个点（ k 从0取到 $N-1$ ），所以完成整个DFT运算总共需要 N^2 次复数乘法及 $N(N-1)$ 次复数加法。在这些运算中乘法运算要比加法运算复杂，需要的运算时间也多一些。因为复数运算实际上是由实数运算来完成的，这时DFT运算式可写成





$$\begin{aligned} X(k) &= \sum_{n=0}^{N-1} x(n) W_N^{nk} = \sum_{n=0}^{N-1} \{ \text{Re}[x(n)] + j \text{Im}[x(n)] \} \{ \text{Re}[W_N^{nk}] + j \text{Im}[W_N^{nk}] \} \\ &= \sum_{n=0}^{N-1} \{ \text{Re}[x(n)] \text{Re}[W_N^{nk}] - \text{Im}[x(n)] \text{Im}[W_N^{nk}] \\ &\quad + j(\text{Re}[x(n)] \text{Im}[W_N^{nk}] + \text{Im}[x(n)] \text{Re}[W_N^{nk}]) \} \end{aligned} \quad (3-3)$$

由此可见，一次复数乘法需用四次实数乘法和二次实数加法；一次复数加法需二次实数加法。因而每运算一个 $X(k)$ 需 $4N$ 次实数乘法和 $2N+2(N-1)=2(2N-1)$ 次实数加法。所以，整个DFT运算总共需要 $4N^2$ 次实数乘法和 $2N(2N-1)$ 次实数加法。





当然，上述统计与实际需要的运算次数稍有出入，因为某些 W_N^{nk} 可能是1或 j ，就不必相乘了，例如 $W_N^0=1$ ， $W_N^{N/2}=-1$ ， $W_N^{N/4}=-j$ 等就不需乘法。但是为了便于和其他运算方法作比较，一般都不考虑这些特殊情况，而是把 W_N^{nk} 都看成复数，**当 N 很大时，这种特例的影响很小。**

从上面的统计可以看到，直接计算DFT，乘法次数和加法次数都是和 N^2 成正比的，当 N 很大时，运算量是很可观的，有时是无法忍受的。





例3-1 根据式（3-1），对一幅 $N \times N$ 点的二维图像进行DFT变换，如用每秒可做10万次复数乘法的计算机，当 $N=1024$ 时，问需要多少时间（不考虑加法运算时间）？

解： 直接计算DFT所需复乘次数为 $(N^2)^2 \approx 10^{12}$ 次，因此用每秒可做10万次复数乘法的计算机，则需要近3000小时。

这对实时性很强的信号处理来说，要么提高计算速度，而这样，对计算速度的要求太高了。另外，只能通过改进对DFT的计算方法，以大大减少运算次数。





3.2.2 改善途径

能否减少运算量，从而缩短计算时间呢？仔细观察DFT的运算就可看出，利用系数 W_N^{nk} 的以下固有特性，就可减少运算量：

(1) W_N^{nk} 的对称性

$$(W_N^{nk})^* = W_N^{-nk}$$

(2) W_N^{nk} 的周期性

$$W_N^{nk} = W_N^{(n+N)k} = W_N^{n(k+N)}$$



(3) W_N^{nk} 的可约性

$$W_N^{nk} = W_{mN}^{nmk}, W_N^{nk} = W_{N/m}^{nk/m}$$

另外

$$W_N^{n(N-k)} = W_N^{(N-n)k} = W_N^{-nk}, W_N^{N/2} = -1, W_N^{(k+N/2)} = -W_N^k$$

这样，利用这些特性，使DFT运算中有些项可以合并，并能使DFT分解为更少点数的DFT运算。而前面已经说到，DFT的运算量是与 N^2 成正比的，所以 N 越小越有利，因而小点数序列的DFT比大点数序列的DFT的运算量要小。

快速傅里叶变换算法正是基于这样的基本思想而发展起来的。它的算法形式有很多种，但基本上可以分成两大类，即按时间抽取（Decimation in Time，缩写为DIT）法和按频率抽取（Decimation in Frequency，缩写为DIF）法。



3.3 按时间抽取（DIT）的基-2 FFT算法

3.3.1 算法原理

设序列 $x(n)$ 长度为 N ，且满足 $N=2^M$ ， M 为正整数。按 n 的奇偶把 $x(n)$ 分解为两个 $N/2$ 点的子序列：

$$\begin{cases} x(2r) = x_1(r) \\ x(2r+1) = x_2(r) \end{cases} \quad r = 0, 1, \dots, \frac{N}{2} - 1 \quad (3-4)$$





则可将DFT化为

$$X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n)W_N^{nk} = \sum_{\substack{n=0 \\ n \text{ 为偶数}}}^{N-1} x(n)W_N^{nk} + \sum_{\substack{n=0 \\ n \text{ 为奇数}}}^{N-1} x(n)W_N^{nk}$$

$$= \sum_{r=0}^{\frac{N}{2}-1} x(2r)W_N^{2rk} + \sum_{r=0}^{\frac{N}{2}-1} x(2r+1)W_N^{(2r+1)k}$$

$$= \sum_{r=0}^{\frac{N}{2}-1} x_1(r)(W_N^2)^{rk} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} x_2(r)(W_N^2)^{rk}$$

由于 $W_N^2 = e^{-j\frac{2\pi}{N}2} = e^{-j2\pi\left(\frac{1}{N/2}\right)} = W_{N/2}$, 故上式可表示成

$$X(k) = \sum_{r=0}^{\frac{N}{2}-1} x_1(r)W_{N/2}^{rk} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} x_2(r)W_{N/2}^{rk} = X_1(k) + W_N^k X_2(k)$$

(3-5)

式中, $X_1(k)$ 与 $X_2(k)$ 分别是 $x_1(r)$ 及 $x_2(r)$ 的 $N/2$ 点DFT:

$$X_1(k) = \sum_{r=0}^{\frac{N}{2}-1} x_1(r) W_{N/2}^{rk} = \sum_{r=0}^{\frac{N}{2}-1} x(2r) W_{N/2}^{rk} \quad (3-6)$$

$$X_2(k) = \sum_{r=0}^{\frac{N}{2}-1} x_2(r) W_{N/2}^{rk} = \sum_{r=0}^{\frac{N}{2}-1} x(2r+1) W_{N/2}^{rk} \quad (3-7)$$

由此, 我们可以看到, 一个 N 点DFT已分解成两个 $N/2$ 点的DFT。这两个 $N/2$ 点的DFT再按照式(3-5)组合成一个 N 点DFT。这里应该看到 $X_1(k)$, $X_2(k)$ 只有 $N/2$ 个点, 即 $k=0, 1, \dots, N/2-1$ 。而 $X(k)$ 却有 N 个点, 即 $k=0, 1, \dots, N-1$, 故用式(3-5)计算得到的只是 $X(k)$ 的前一半的结果, 要用 $X_1(k)$, $X_2(k)$ 来表达全部的 $X(k)$ 值, 还必须应用系数的周期性, 即



$$W_{N/2}^{r\left(k+\frac{N}{2}\right)} = W_{N/2}^{rk}$$

这样可得到

$$X_1\left(\frac{N}{2} + k\right) = \sum_{r=0}^{\frac{N}{2}-1} x_1(r) W_{N/2}^{r\left(\frac{N}{2}+k\right)} = \sum_{r=0}^{\frac{N}{2}-1} x_1(r) W_{N/2}^{rk} = X_1(k) \quad (3-8)$$

同理可得

$$X_2\left(\frac{N}{2} + k\right) = X_2(k) \quad (3-9)$$

式 (3-8)、式 (3-9) 说明了后半部分 k 值 ($N/2 \leq k \leq N-1$) 所对应的 $X_1(k)$, $X_2(k)$ 分别等于前半部分 k 值 ($0 \leq k \leq N/2-1$) 所对应的 $X_1(k)$, $X_2(k)$ 。





再考虑到 W_N^k 的以下性质:

$$W_N^{\left(\frac{N}{2}+k\right)} = W_N^{N/2} W_N^k = -W_N^k \quad (3-10)$$

这样, 把式(3-8)、式(3-9)、式(3-10)代入式(3-5), 就可将 $X(k)$ 表达为前后两部分:

$$X(k) = X_1(k) + W_N^k X_2(k) \quad k = 0, 1, \dots, \frac{N}{2} - 1 \quad (3-11)$$

$$\begin{aligned} X\left(k + \frac{N}{2}\right) &= X_1\left(k + \frac{N}{2}\right) + W_N^{\left(k + \frac{N}{2}\right)} X_2\left(k + \frac{N}{2}\right) \\ &= X_1(k) - W_N^k X_2(k) \quad k = 0, 1, \dots, \frac{N}{2} - 1 \end{aligned} \quad (3-12)$$



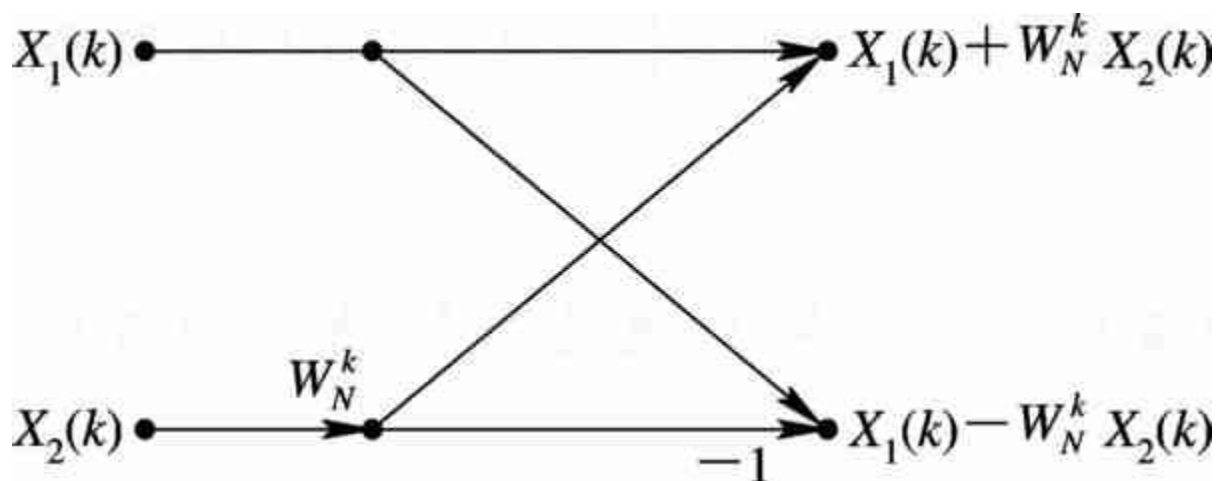


图 3-1 时间抽取法蝶形运算流图符号

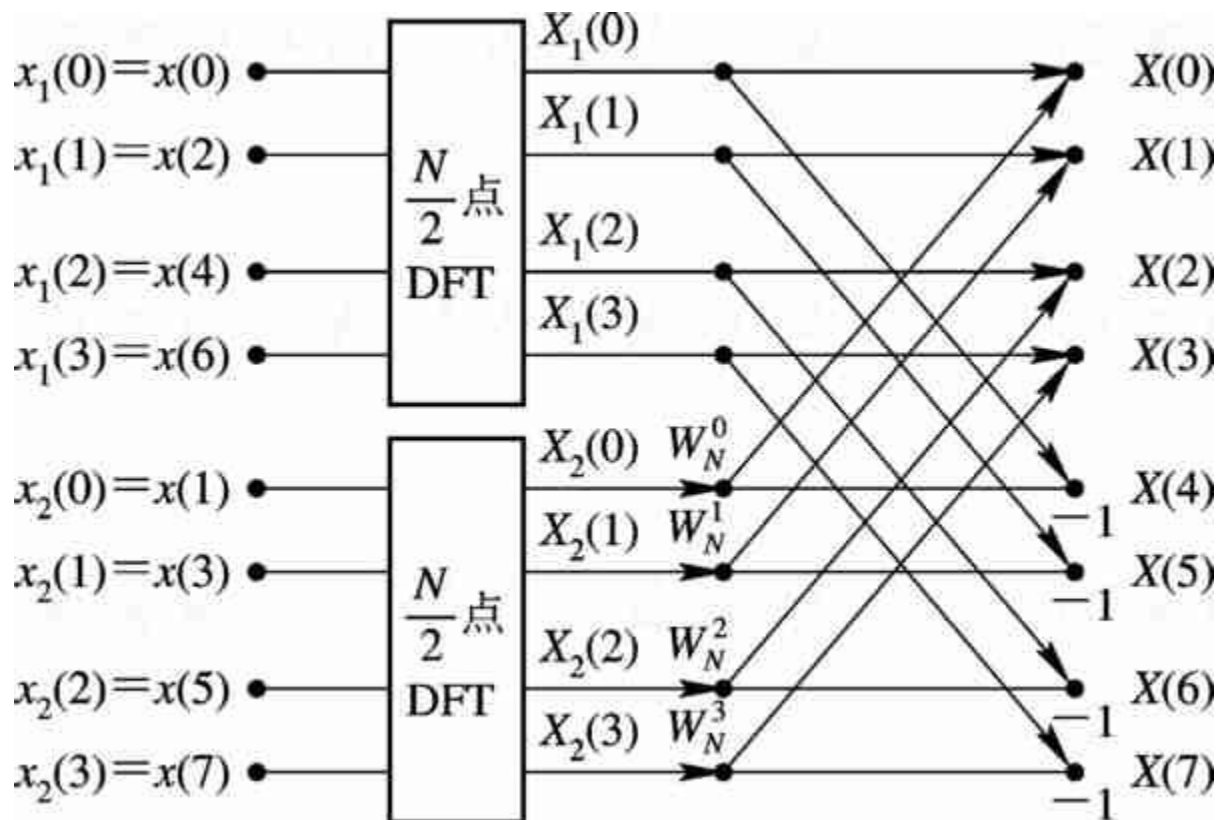


图 3-2 按时间抽取将一个 N 点DFT分解为两个 $N/2$ 点DFT ($N=8$)



一个 N 点DFT分解为两个 $N/2$ 点DFT，每一个 $N/2$ 点DFT只需 $(N/2)^2=N^2/4$ 次复数乘法， $N/2(N/2-1)$ 次复数加法。两个 $N/2$ 点DFT共需 $2 \times (N/2)^2=N^2/2$ 次复数乘法和 $N(N/2-1)$ 次复数加法。此外，把两个 $N/2$ 点DFT合成为 N 点DFT时，有 $N/2$ 个蝶形运算，还需要 $N/2$ 次复数乘法及 $2 \times N/2=N$ 次复数加法。因而通过第一步分解后，总共需要 $(N^2/2)+(N/2)=N(N+1)/2 \approx N^2/2$ 次复数乘法和 $N(N/2-1)+N=N^2/2$ 次复数加法。由此可见，通过这样分解后运算工作量差不多节省了一半。

既然这样分解是有效的，由于 $N=2^M$ ，因而 $N/2$ 仍是偶数，可以进一步把每个 $N/2$ 点子序列再按其奇偶部分分解为两个 $N/4$ 点的子序列。





$$\begin{cases} x_1(2l) = x_3(l) \\ x_1(2l+1) = x_4(l) \end{cases} \quad l = 0, 1, \dots, \frac{N}{4} - 1 \quad (3-13)$$

$$X_1(k) = \sum_{l=0}^{\frac{N}{4}-1} x_1(2l) W_{N/2}^{2lk} + \sum_{l=0}^{\frac{N}{4}-1} x_1(2l+1) W_{N/2}^{(2l+1)k}$$

$$= \sum_{l=0}^{\frac{N}{4}-1} x_3(l) W_{N/4}^{lk} + W_{N/2}^k \sum_{l=0}^{\frac{N}{4}-1} x_4(l) W_{N/4}^{lk}$$

$$= X_3(k) + W_{N/2}^k X_4(k) \quad k = 0, 1, \dots, \frac{N}{4} - 1$$





且

$$X_1\left(\frac{N}{4} + k\right) = X_3(k) - W_{N/2}^k X_4(k) \quad k = 0, 1, \dots, \frac{N}{4} - 1$$

式中：

$$X_3(k) = \sum_{l=0}^{\frac{N}{4}-1} x_3(l) W_{N/4}^{lk} \quad (3-14)$$

$$X_4(k) = \sum_{l=0}^{\frac{N}{4}-1} x_4(l) W_{N/4}^{lk} \quad (3-15)$$

图3-3给出 $N=8$ 时，将一个 $N/2$ 点DFT分解成两个 $N/4$ 点DFT，由这两个 $N/4$ 点DFT组合成一个 $N/2$ 点DFT的流图。



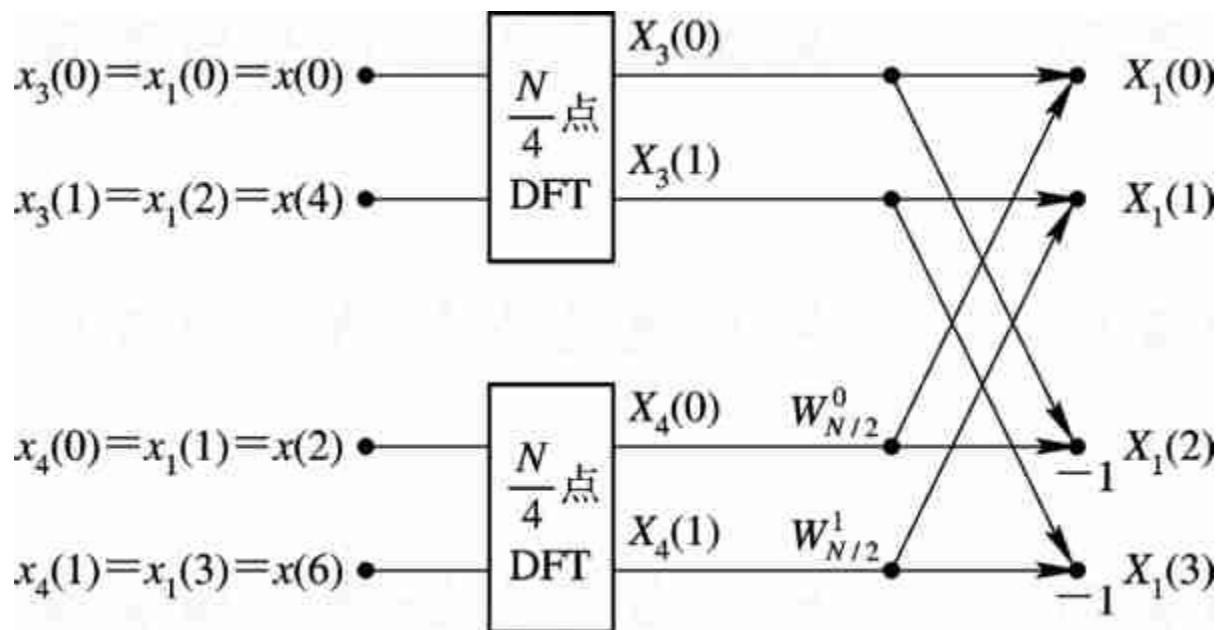


图 3-3 $N/2$ 点DFT分解为两个 $N/4$ 点DFT



$X_2(k)$ 也可进行同样的分解:

$$\begin{cases} X_2(k) = X_5(k) + W_{N/2}^k X_6(k) \\ X_2\left(\frac{N}{4} + k\right) = X_5(k) - W_{N/2}^k X_6(k) \end{cases} \quad k = 0, 1, \dots, \frac{N}{4} - 1$$

式中:

$$X_5(k) = \sum_{l=0}^{\frac{N}{4}-1} x_5(l) W_{N/4}^{lk} = \sum_{l=0}^{\frac{N}{4}-1} x_2(2l) W_{N/4}^{lk} \quad (3-16)$$

$$X_6(k) = \sum_{l=0}^{\frac{N}{4}-1} x_6(l) W_{N/4}^{lk} = \sum_{l=0}^{\frac{N}{4}-1} x_2(2l+1) W_{N/4}^{lk} \quad (3-17)$$



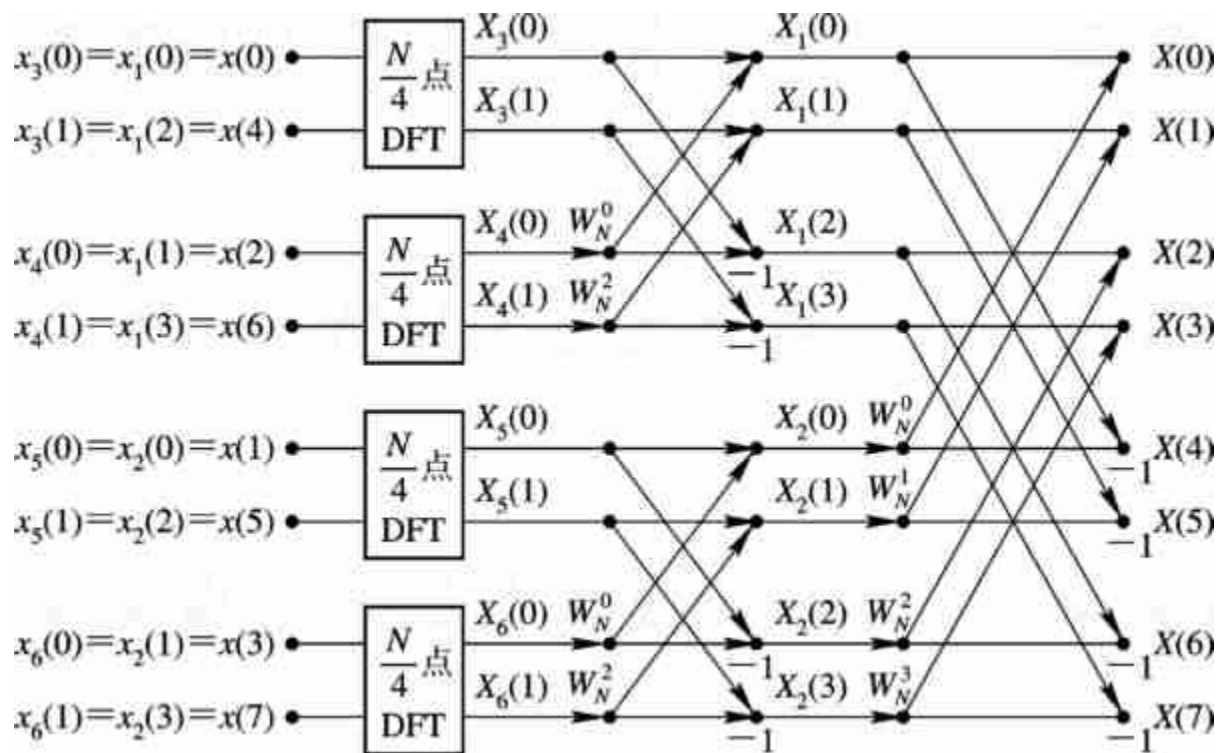


图 3-4 一个 $N=8$ 点 DFT 分解为四个 $N/4$ 点 DFT

根据上面同样的分析知道，利用四个 $N/4$ 点的DFT及两级蝶形组合运算来计算 N 点DFT，比只用一次分解蝶形组合方式的计算量又减少了大约一半。

最后，剩下的是2点DFT，对于此例 $N=8$ ，就是四个 $N/4=2$ 点DFT，其输出为 $X_3(k)$, $X_4(k)$, $X_5(k)$, $X_6(k)$, $k=0, 1$ ，这由式（3-14）～式（3-17）四个式子可以计算出来。例如，由式（3-14）可得：

$$X_3(0) = x_3(0) + W_2^0 x_3(1) = x(0) + W_2^0 x(4) = x(0) + W_N^0 x(4)$$

$$X_3(1) = x_3(0) + W_2^1 x_3(1) = x(0) + W_2^1 x(4) = x(0) - W_N^0 x(4)$$

式中， $W_2^1 = e^{-j\frac{2\pi}{2} \times 1} = e^{-j\pi} = -1 = -W_N^0$ ，故上式不需要乘法。类似地，可求出 $X_4(k)$, $X_5(k)$, $X_6(k)$ 。

这种方法的每一步分解，都是按输入序列在时间上的次序是属于偶数还是属于奇数来分解为两个更短的子序列，所以称为“按时间抽取法”。

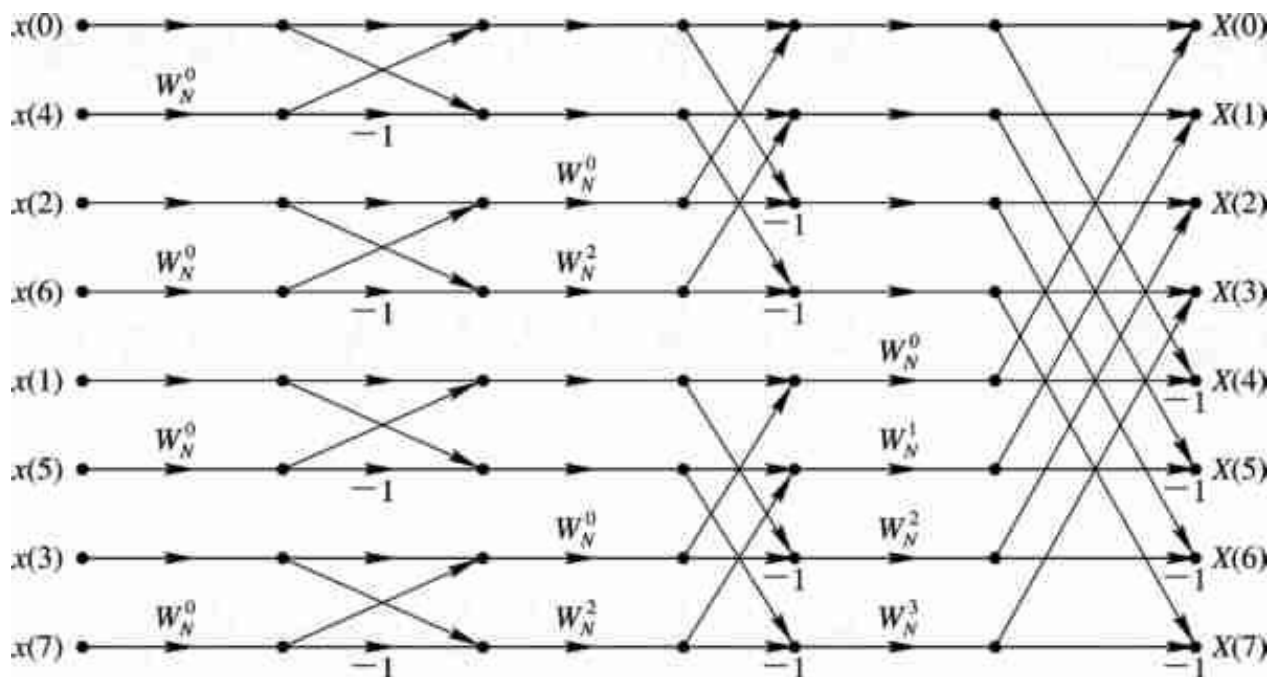


图 3-5 $N=8$ 按时间抽取的FFT运算流图



3.3.2 按时间抽取的FFT算法与直接计算DFT运算量的比较

由按时间抽取法FFT的流图可见，当 $N=2^M$ 时，共有 M 级蝶形，每级都由 $N/2$ 个蝶形运算组成，每个蝶形需要一次复乘、二次复加，因而每级运算都需 $N/2$ 次复乘和 N 次复加，这样 M 级运算总共需要

$$\text{复乘数} \quad m_F = \frac{N}{2} M = \frac{N}{2} 1bN \quad (3-18)$$

$$\text{复加数} \quad a_F = NM = N1bN \quad (3-19)$$

式中，数学符号 $1b = \log_2$ 。





实际计算量与这个数字稍有不同，因为 $W_N^0=1$ ， $W_N^{N/2}=-1$ ，

$W_N^{N/4}=-j$ ，这几个系数都不用乘法运算，但是这些情况在直接计算DFT中也是存在的。此外，当 N 较大时，这些特例相对而言就很少。所以，为了统一作比较起见，下面都不考虑这些特例。

由于计算机上乘法运算所需的时间比加法运算所需的时间多得多，故以乘法为例，直接DFT复数乘法次数是 N^2 ，FFT复数乘法次数是 $(N/2) \lg N$ 。直接计算DFT与FFT算法的计算量之比为

$$\frac{\frac{N^2}{2}}{\frac{N}{2} \lg N} = \frac{N^2}{N \lg N} = \frac{2N}{\lg N} \quad (3-20)$$

当 $N=2048$ 时，这一比值为372.4，即直接计算DFT的运算量是FFT运算量的372.4倍。当点数 N 越大时，FFT的优点更为明显。





例3-2 用FFT算法处理一幅 $N \times N$ 点的二维图像，如用每秒可做10万次复数乘法的计算机，当 $N=1024$ 时，问需要多少时间（不考虑加法运算时间）？

解 当 $N=1024$ 点时，FFT算法处理一幅二维图像所需复数乘法约为 $\frac{N^2}{2} \lg N \approx 10^7$ 次，仅为直接计算DFT所需时间的10万分之一。即原需要3000小时，现在只需要2 分钟。





3.3.3 按时间抽取的FFT算法的特点及DIT-FFT程序框图

1. 原位运算（同址运算）

从图3-5可以看出这种运算是很有规律的，其每级（每列）计算都是由 $N/2$ 个蝶形运算构成的，每一个蝶形结构完成下述基本迭代运算：

$$X_m(k) = X_{m-1}(k) + X_{m-1}(j)W_N^r \quad (3-21a)$$

$$X_m(j) = X_{m-1}(k) - X_{m-1}(j)W_N^r \quad (3-21b)$$

式中， m 表示第 m 列迭代， k, j 为数据所在行数。式（3-21）的蝶形运算如图3-6所示，由一次复乘和两次复加（减）组成。



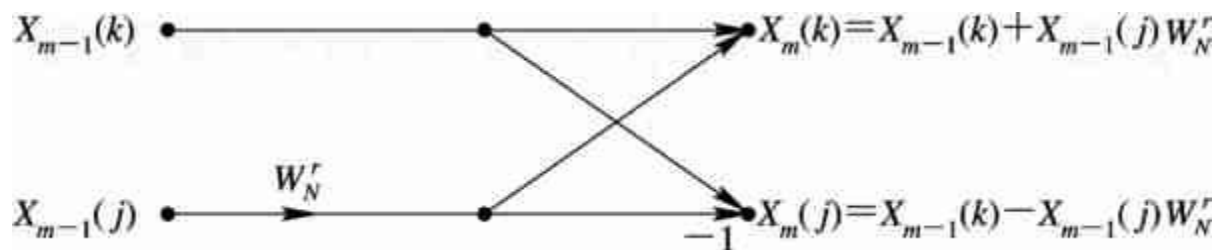


图 3-6 蝶形运算单元



由图3-5的流图看出，某一系列的任何两个节点 k 和 j 的节点变量进行蝶形运算后，得到结果为下一列 k, j 两节点的节点变量，而和其他节点变量无关，因而可以采用原位运算，即某一系列的 N 个数据送到存储器后，经蝶形运算，其结果为下一列数据，它们以蝶形为单位仍存储在这同一组存储器中，直到最后输出，中间无需其他存储器。也就是蝶形的两个输出值仍放回蝶形的两个输入所在的存储器中。每列的 $N/2$ 个蝶形运算全部完成后，再开始下一列的蝶形运算。这样存储器数据只需 N 个存储单元。下一级的运算仍采用这种原位方式，只不过进入蝶形结的组合关系有所不同。这种原位运算结构可以节省存储单元，降低设备成本。





2. 倒位序规律

观察图3-5的同址计算结构，发现当运算完成后，FFT的输出 $X(k)$ 按正常顺序排列在存储单元中，即按 $X(0)$ ， $X(1)$ ，...， $X(7)$ 的顺序排列，但是这时输入 $x(n)$ 却不是按自然顺序存储的，而是按 $x(0)$ ， $x(4)$ ，...， $x(7)$ 的顺序存入存储单元，看起来好像是“混乱无序”的，实际上是有规律的，我们称之为倒位序。





造成倒位序的原因是输入 $x(n)$ 按标号 n 的偶奇的不断分组。如果 n 用二进制数表示为 $(n_2n_1n_0)_2$ （当 $N=8=2^3$ 时，二进制为三位），第一次分组，由图3-2看出， n 为偶数（相当于 n 的二进制数的最低位 $n_0=0$ ）在上半部分， n 为奇数（相当于 n 的二进制数的最低位 $n_0=1$ ）在下半部分。下一次则根据次最低位 n_1 为“0”或是“1”来分偶奇（而不管原来的子序列是偶序列还是奇序列），如此继续分下去，直到最后 N 个长度为1的子序列。图3-7的树状图描述了这种分成偶数子序列和奇数子序列的过程。



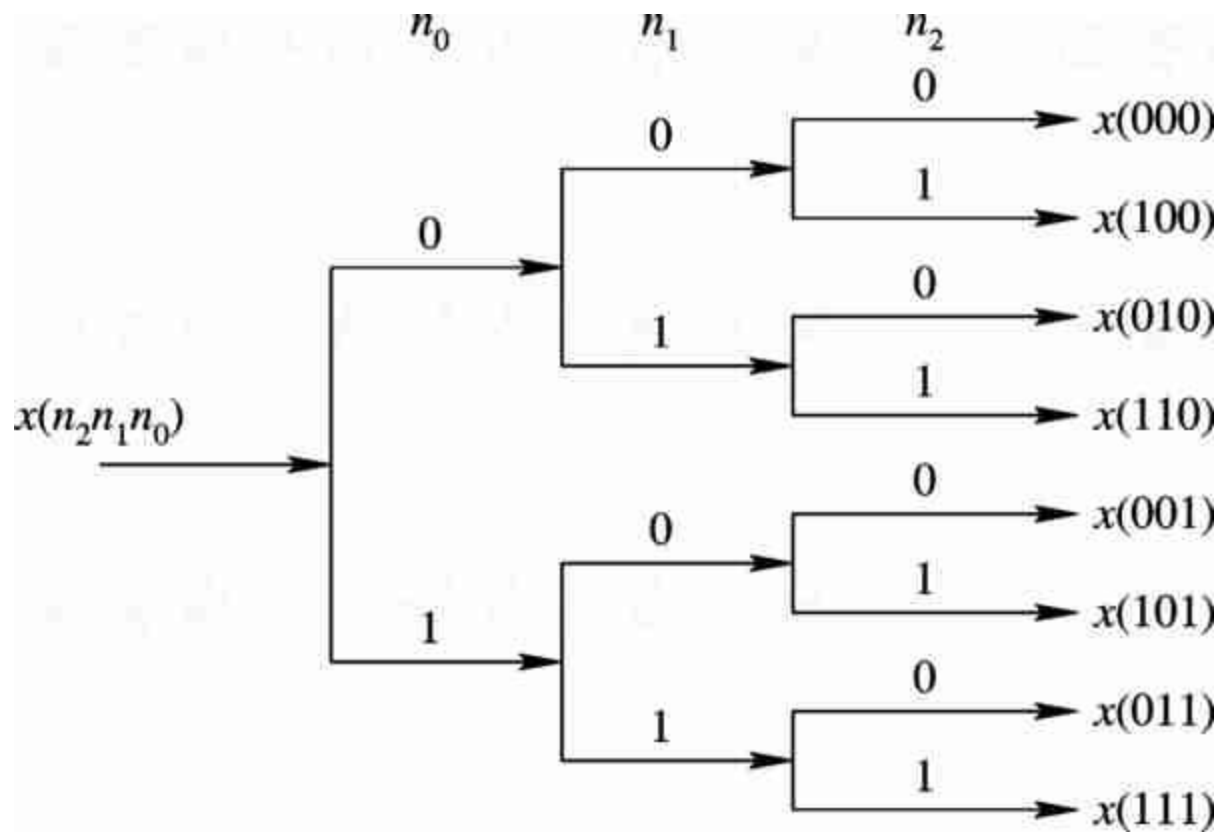


图3-7 倒位序的形成



一般实际运算中，总是先按自然顺序将输入序列存入存储单元，为了得到倒位序的排列，我们通过变址运算来完成。

如果输入序列的自然顺序号 I 用二进制数（例如 $n_2n_1n_0$ ）表示，则其倒位序 J 对应的二进制数就是 $(n_0n_1n_2)$ ，这样，在原来自然顺序时应该放 $x(I)$ 的单元，现在倒位序后应放 $x(J)$ 。例如， $N=8$ 时， $x(3)$ 的标号是 $I=3$ ，它的二进制数是011，倒位序的二进制数是110，即 $J=6$ ，所以原来存放在 $x(011)$ 单元的数据现在应该存放在 $x(110)$ 内。表3-1列出了 $N=8$ 时的自然顺序二进制数以及相应的倒位序二进制数。





表3-1 N=8时的自然顺序二进制数和相应的倒位序二进制数

自然顺序 (I)	二进制数	倒位序二进制数	倒位序 (J)
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7





由表3-1可见，自然顺序数 I 增加1，是在顺序数的二进制数最低位加1，向左进位。而倒序数 J 则是在二进制数最高位加1，逢2向右进位。例如，在(000)最高位加1，则得(100)，再在(100)最高位加1，向右进位，则得(010)。因(100)最高位为1，所以最高位加1要向次高位进位，其实质是将最高位变为0，再在次高位加1。用这种算法，可以从当前任一倒序值求得下一个倒序值。





对于 $N=2^M$ ， M 为二进制数最高位，其权值为 $N/2$ ，且从左向右二进制位的权值依次为 $N/4$ ， $N/8$ ，...，2，1。因此，最高位加1相当于十进制运算 $J+N/2$ 。如果最高位是0（ $J < N/2$ ），则直接由 $J+N/2$ 得下一个倒序值；如果最高位是1（ $J \geq N/2$ ），则要将最高位变为0（ $J \leftarrow J - N/2$ ），次高位加1（ $J+N/4$ ）。但次高位加1时，同样要判断次高位0、1值，如果为0（ $J < N/4$ ），则直接加1（ $J \leftarrow J+N/4$ ）；否则，将次高位变为0（ $J \leftarrow J - N/4$ ），再判断下一位；以此类推，直到完成最高位加1，逢2向右进位的运算。





把按自然顺序存放在存储单元中的数据，换成FFT原位运算流程图所要求的倒位序的变址功能如图3-8所示，当 $I=J$ 时，不必调换，当 $I \neq J$ 时，必须将原来存放数据 $x(I)$ 的存储单元内调入数据 $x(J)$ ，而将存放 $x(J)$ 的存储单元内调入 $x(I)$ 。为了避免把已调换过的数据再次调换，保证只调换一次（否则又回到原状），我们只需看 J 是否比 I 小。若 J 比 I 小，则意味着此 $x(I)$ 在前边已和 $x(J)$ 互相调换过，不必再调换了；只有当 $J > I$ 时，才将原存放 $x(I)$ 及存放 $x(J)$ 的存储单元内的内容互换。这样就得到输入所需的倒位序列的顺序。可以看出，其结果与图3-5的要求是一致的。



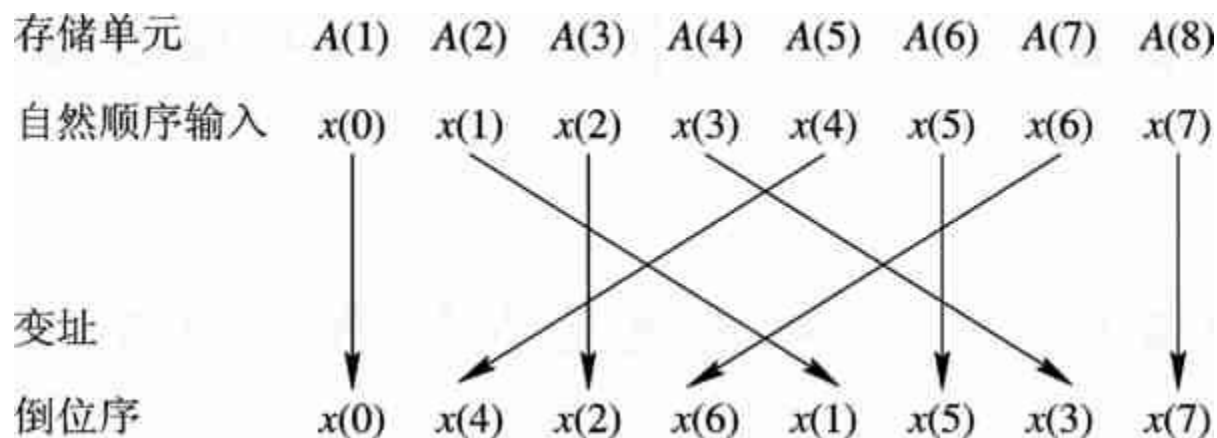


图 3-8 $N=8$ 倒位序的变址处理



3. 蝶形运算两节点的“距离”

以图3-5的8点FFT为例，其输入是倒位序的，输出是自然顺序的。其第一级（第一列）每个蝶形的两节点间“距离”为1，第二级每个蝶形的两节点“距离”为2，第三级每个蝶形的两节点“距离”为4。由此类推得，对 $N=2^M$ 点FFT，当输入为倒位序，输出为正常顺序时，其第 m 级运算，每个蝶形的两节点“距离”为 2^{m-1} 。





4. W_N^r 的确定

由于对第 m 级运算，一个DFT蝶形运算的两节点“距离”为 2^{m-1} ，因而式（3-21）可写成：

$$X_m(k) = X_{m-1}(k) + X_{m-1}(k + 2^{m-1})W_N^r \quad (3-22a)$$

$$X_m(k + 2^{m-1}) = X_{m-1}(k) - X_{m-1}(k + 2^{m-1})W_N^r \quad (3-22b)$$

为了完成上两式运算，还必须知道系数 W_N^r 的变换规律。仔细观察图3-5的流图可以发现 r 的变换规律是：

① 把式（3-22）中蝶形运算两节点中的第一个节点标号值，即 k 值，表示成 M 位（注意 $N=2^M$ ）二进制数；





② 把此二进制数乘上 2^{M-m} ，即将此 M 位二进制数左移 $M-m$ 位（注意 m 是第 m 级运算），把右边空出的位置补零，此数即为所求 r 的二进制数。

从图3-5看出， W_N^r 因子最后一列有 $N/2$ 种，顺序为 W_N^0 ， W_N^1 ，...， $W_N^{\left(\frac{N}{2}-1\right)}$ ，其余可类推。



3.4 利用FFT分析时域连续信号频谱

3.4.1 基本步骤

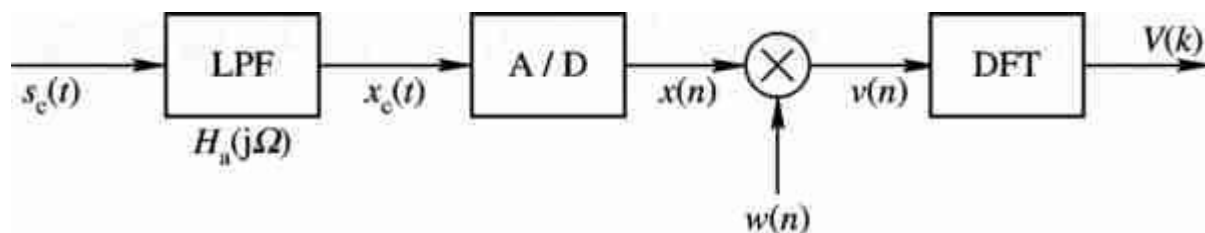


图 3-21 时域连续信号离散傅里叶分析的处理步骤

在图3-21中，前置低通滤波器LPF（预滤波器）的引入，是为了消除或减少时域连续信号转换成序列时可能出现的频谱混叠的影响。在实际工作中，时域离散信号 $x(n)$ 的时宽是很长的，甚至是无限长的（例如语音或音乐信号）。由于DFT的需要(实际应用FFT计算)，必须把 $x(n)$ 限制在一定的时间区间之内，即进行数据截断。数据的截断相当于加窗处理（窗函数见6.2节）。因此，在计算FFT之前，**用一个时域有限的窗函数 $w(n)$ 加到 $x(n)$ 上是非常必要的。**

$x_c(t)$ 通过A/D变换器转换(忽略其幅度量化误差)成采样序列 $x(n)$ ，其频谱用 $X(e^{j\omega})$ 表示，它是频率 ω 的周期函数，即

$$X(e^{j\omega}) = \frac{1}{T} \sum_{m=-\infty}^{\infty} X_c\left(j\frac{\omega}{T} - j\frac{2\pi m}{T}\right) \quad (3-47)$$

式中， $X_c(j\Omega)$ 或 $X_c\left(j\frac{\omega}{T}\right)$ 为 $x_c(t)$ 的频谱。



在实际应用中，前置低通滤波器的阻带不可能是无限衰减的，故由 $X_c(j\Omega)$ 周期延拓得到的 $X(e^{j\omega})$ 有非零重叠，即出现频谱混叠现象。

由于进行FFT的需要，必须对序列 $x(n)$ 进行加窗处理，即 $v(n)=x(n)w(n)$ ，加窗对频域的影响，用卷积表示如下：

$$V(e^{j\omega}) = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(e^{j\theta}) W(e^{j(\omega-\theta)}) d\theta \quad (3-48)$$

最后是进行FFT运算。加窗后的DFT是

$$V(k) = \sum_{n=0}^{N-1} v(n) e^{-j\frac{2\pi}{N}nk} \quad 0 \leq k \leq N-1 \quad (3-49)$$

式中，假设窗函数长度 L 小于或等于DFT长度 N ，为进行FFT运算，这里选择 N 为2的整数幂次即 $N=2^m$ 。





有限长序列 $v(n)=x(n)w(n)$ 的DFT相当于 $v(n)$ 傅里叶变换的等间隔采样。

$$V(k) = V(e^{j\omega}) \Big|_{\omega = \frac{2\pi}{N}k} \quad (3-50)$$

$V(k)$ 便是 $s_c(t)$ 的离散频率函数。因为DFT对应的数字域频率间隔为 $\Delta\omega=2\pi/N$ ，且模拟频率 Ω 和数字频率 ω 间的关系为 $\omega=\Omega T$ ，其中 $\Omega=2\pi f$ 。所以，离散的频率函数第 k 点对应的模拟频率为：

$$\Omega_k = \frac{\omega}{T} = \frac{2\pi k}{NT} \quad (3-51)$$

$$f_k = \frac{k}{NT} \quad (3-52)$$





由式（3-52）很明显可看出，数字域频率间隔 $\Delta\omega=2\pi/N$ 对应的模拟域谱线间距为

$$F = \frac{1}{NT} = \frac{f_s}{N} \quad (3-53)$$

谱线间距，又称频谱分辨率（单位：Hz）。所谓频谱分辨率是指可分辨两频率的最小间距。它的意思是，如设某频谱分析的 $F=5\text{Hz}$ ，那么信号中频率相差小于5 Hz的两个频率分量在此频谱图中就分辨不出来。





长度 $N=16$ 的时间信号 $v(n)=(1.1)^n R_{16}(n)$ 的图形如图 3-22(a)所示, 其16点的DFT $V(k)$ 的示例如图 3 - 22(b)所示。 其中 T 为采样时间间隔 (单位: s) ; f_s 为采样频率 (单位: Hz) ; t_p 为截取连续时间信号的样本长度 (又称记录长度, 单位: s) ; F 为谱线间距, 又称频谱分辨率 (单位: Hz) 。 注意: $V(k)$ 示例图给出的频率间距 F 及 N 个频率点之间的频率 f_s 为对应的模拟域频率(单位: Hz) 。



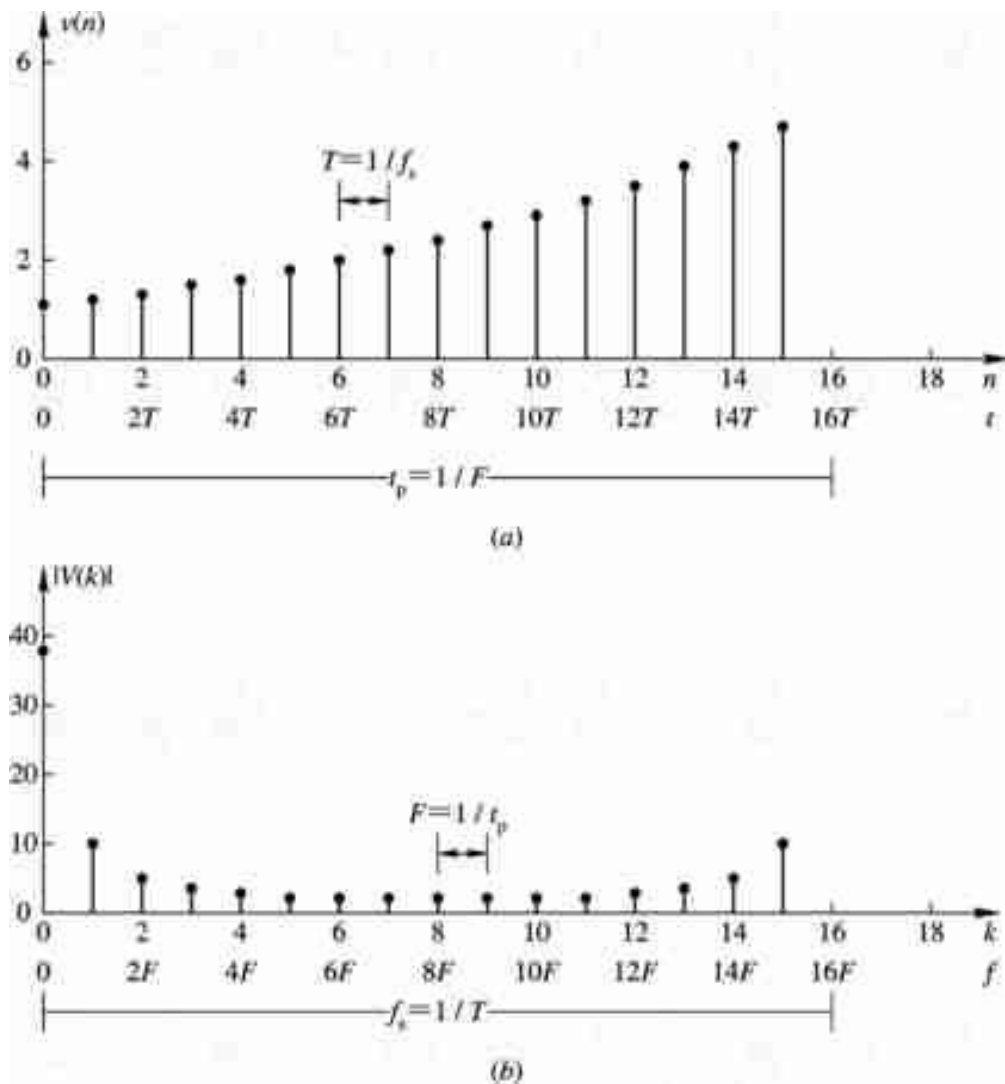


图3-22 $v(n)=(1.1)^n R_{16}(n)$ 及DFT $V(k)$ 示例图



由图可知:

$$t_p = NT \quad (3-54)$$

$$F = \frac{f_s}{N} = \frac{1}{NT} = \frac{1}{t_p} \quad (3-55)$$

在实际应用中, 要根据信号最高频率 f_h 和频谱分辨率 F 的要求, 来确定 T 、 t_p 和 N 的大小。

(1) 首先, 由采样定理, 为保证采样信号不失真, $f_s \geq 2f_h$ (f_h 为信号频率的最高频率分量, 也就是前置低通滤波器阻带的截止频率), 即应使采样周期 T 满足

$$T \leq \frac{1}{2f_h} \quad (3-56)$$





(2) 由频谱分辨率 F 和 T 确定 N ,

$$N = \frac{f_s}{F} = \frac{1}{FT} \quad (3-57)$$

为了使用FFT运算, 这里选择 N 为2的幂次即 $N=2^m$, 由式(3-55)可知, N 大, 分辨率好, 但会增加样本记录时间 t_p 。

(3) 最后由 N , T 确定最小记录长度, $t_p=NT$ 。





[例3-3] 有一频谱分析用的FFT处理器，其采样点数必须是2的整数幂，假定没有采用任何特殊的数据处理措施，已给条件为：

① 频率分辨率 $\leq 10\text{ Hz}$ ； ② 信号最高频率 $\leq 4\text{ kHz}$ 。试确定以下参量：

- (1) 最小记录长度 t_p ；
- (2) 最大采样间隔 T （即最小采样频率）；
- (3) 在一个记录中的最少点数 N 。





解

(1) 由分辨率的要求确定最小长度 t_p

$$\frac{1}{F} = \frac{1}{10} = 0.1s$$

所以记录长度为

$$t_p \geq \frac{1}{F} = 0.1s$$





(2) 从信号的最高频率确定最大可能的采样间隔 T （即最小采样频率 $f_s=1/T$ ）。按采样定理

$$f_s \geq 2f_h$$

即

$$T \leq \frac{1}{2f_h} = \frac{1}{2 \times 4 \times 10^3} = 0.125 \times 10^{-3} s$$





(3) 最小记录点数 N 应满足

$$N > \frac{2f_h}{F} = \frac{2 \times 4 \times 10^3}{10} = 800$$

取

$$N = 2^m = 2^{10} = 1024 > 800$$





如果我们事先不知道信号的最高频率，可以根据信号的时域波形图来估计它。例如，某信号的波形如图 3-23 所示。先找出相邻的波峰与波谷之间的距离，如图中 t_1 , t_2 , t_3 , t_4 。然后，选出其中最小的一个如 t_4 。这里， t_4 可能就是由信号的最高频率分量形成的。峰与谷之间的距离就是周期的一半。因此，最高频率为

$$f_h = \frac{1}{2t_4} \quad (\text{Hz})$$

知道 f_h 后就能确定采样频率

$$f_s > 2f_h$$



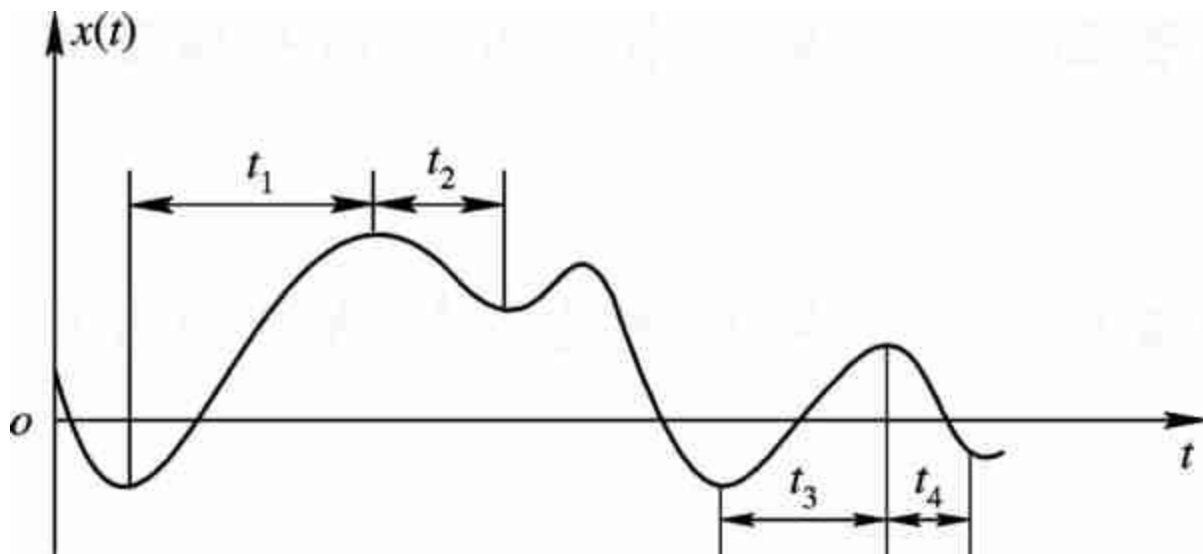


图 3-23 估算信号最高频率 f_h



3.4.2 可能出现的误差

1. 频谱混叠失真

在图3-21 画出的基本步骤中，A/D变换前利用前置低通滤波器进行预滤波，使 $x_c(t)$ 频谱中最高频率分量不超过 f_h 。假设A/D变换器的采样频率为 f_s ，按照奈奎斯特采样定理，为了不产生混叠，必须满足

$$f_s \geq 2f_h$$

也就是采样间隔 T 满足

$$T = \frac{1}{f_s} \leq \frac{1}{2f_h}$$





一般应取

$$f_s = (2.5 \sim 3.0)f_h \quad (3-58)$$

如果不满足 $f_s \geq 2f_h$ ，就会产生频谱混叠失真。

对于FFT来说，频率函数也要采样，变成离散的序列，其采样间隔为 F (即频率分辨率)。由式(3-55)可得

$$t_p = \frac{1}{F} \quad (3-59)$$

从以上 T 和 t_p 两个公式来看，信号的最高频率分量 f_h 与频率分辨率 F 存在矛盾关系，要想 f_h 增加，则时域采样间隔 T 就一定减小，而 f_s 就增加，由式(3-57)可知，此时若是固定 N ，必然要增加 F ，即分辨率下降。



反之，要提高分辨率（减小 F ），就要增加 t_p ，当 N 给定时，必然导致 T 的增加（ f_s 减小）。要不产生混叠失真，则必然会减小高频容量（信号的最高频率分量） f_h 。

要想兼顾高频容量 f_h 与频率分辨率 F ，即一个性能提高而另一个性能不变（或也得以提高）的惟一办法就是增加记录长度的点数 N ，即要满足

$$N = \frac{f_s}{F} > \frac{2f_h}{F} \quad (3-60)$$

这个公式是未采用任何特殊数据处理（例如加窗处理）的情况下，为实现基本FFT算法所必须满足的最低条件。如果加窗处理，相当于时域相乘，则频域周期卷积，必然加宽频谱分量，频率分辨率就可能变差，为了保证频率分辨率不变，则须增加数据长度 t_p 。



2. 栅栏效应

利用FFT计算频谱，只给出离散点 $\omega_k=2\pi k/N$ 或 $\Omega_k=2\pi k/(NT)$ 上的频谱采样值，而不可能得到连续频谱函数，这就像通过一个“栅栏”观看信号频谱，只能在离散点上看到信号频谱，称之为“栅栏效应”，如图3-22所示。这时，如果在两个离散的谱线之间有一个特别大的频谱分量，就无法检测出来了。





减小栅栏效应的一个方法就是要使频域采样更密，即增加频域采样点数 N ，在不改变时域数据的情况下，必然是在数据末端添加一些零值点，使一个周期内的点数增加，但并不改变原有的记录数据。频谱采样为 $2\pi k/N$ ， N 增加，必然使样点间距更近（单位圆上样点更多），谱线更密，谱线变密后原来看不到的谱分量就有可能看到了。

必须指出，补零以改变计算FFT的周期时，所用窗函数的宽度不能改变。换句话说，必须按照数据记录的原来的实际长度选择窗函数，而不能按照补了零值点后的长度来选择窗函数。

补零不能提高频率分辨率，这是因为数据的实际长度仍为补零前的数据长度。





3. 频谱泄漏与谱间干扰

对信号进行FFT计算，首先必须使其变成有限时宽的信号，这就相当于信号在时域乘一个窗函数如矩形窗，窗内数据并不改变。时域相乘即 $v(n)=x(n) \cdot w(n)$ ，加窗对频域的影响，可用式（3-48）卷积公式表示

$$V(e^{j\omega}) = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(e^{j\theta}) W(e^{j(\omega-\theta)}) d\theta$$

卷积的结果，造成所得到的频谱 $V(e^{j\omega})$ 与原来的频谱 $X(e^{j\omega})$ 不相同，有失真。这种失真最主要的是造成频谱的“扩散”（拖尾、变宽），这就是所谓的“频谱泄漏”。





由上可知，**泄漏是由于我们截取有限长信号所造成的**。对具有单一谱线的正弦波来说，它必须是无限长的。也就是说，如果我们输入信号是无限长的，那么FFT就能计算出完全正确的单一线频谱。可是我们不可能这么做，而只能取有限长记录样本。如果在该有限长记录样本中，正弦信号又不是整数个周期时，就会产生泄漏。例如，一个周期为 $N=16$ 的余弦信号 $x(n)=\cos(6\pi n / 16)$ 截取一个周期长度的信号即 $x_1(n)=\cos(6\pi n/16)R_{16}(n)$, 其16点FFT的谱图见 3-24上图，若截取的长度为13，则其16点FFT的谱图见3-24下图。由此可见，频谱不再是单一的谱线，它的能量散布到整个频谱的各处。这种能量散布到其他谱线位置的现象即为“频谱泄漏”。





应该说明，泄漏也会造成混叠，因为泄漏将会导致频谱的扩展，从而使最高频率有可能超过折叠频率 ($f_s/2$)，造成频率响应的混叠失真。

泄漏造成的后果是降低频谱的分辨率。此外，由于在主谱线两边形成很多旁瓣，引起不同频率分量间的干扰(简称谱间干扰)，特别是强信号谱的旁瓣可能湮没弱信号的主谱线，或者把强信号谱的旁瓣误认为是另一信号的谱线，从而造成假信号，这样就会使谱分析产生较大偏差。





在进行FFT运算时，时域截断是必然的，从而频谱泄漏和谱间干扰也是不可避免的。为尽量减小泄漏和谱间干扰的影响，需增加窗的时域宽度(频域主瓣变窄)，但这又导致运算量及存储量的增加；

其次，数据不要突然截断，也就是不要加矩形窗，而是加各种缓变的窗（例如：三角形窗、升余弦窗、改进的升余弦窗等），使得窗谱的旁瓣能量更小，卷积后造成的泄漏减小，这个问题在FIR滤波器设计一章（第6章）中将会讨论到。



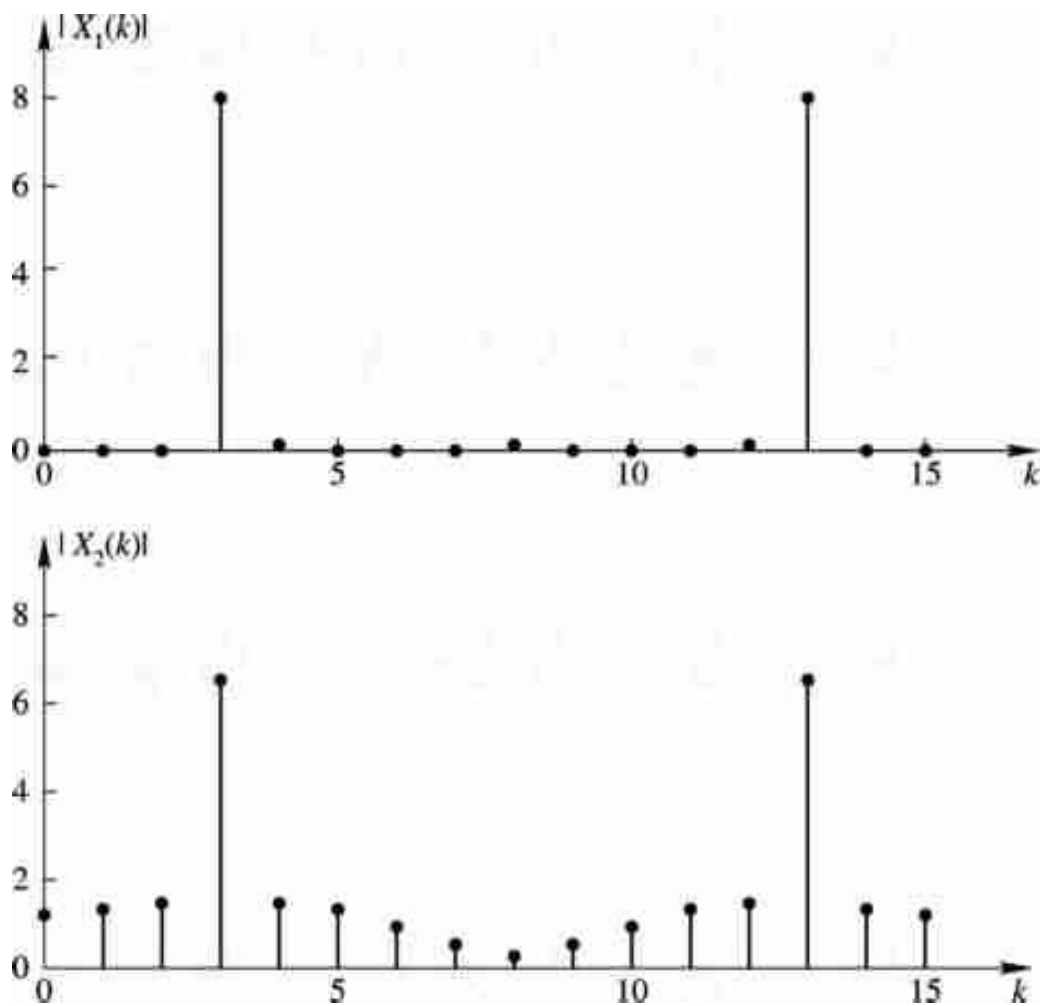


图 3-24 余弦信号频谱泄漏示例图



3.4.3 应用实例

一个长度为 N 的时域离散序列 $x(n)$ ，其离散傅里叶变换 $X(k)$ （离散频谱）是由实部和虚部组成的复数，即

$$X(k) = X_R(k) + jX_I(k) \quad (3-61)$$

对实信号 $x(n)$ ，其频谱是共轭偶对称的，故只要求出 k 在 $0, 1, 2, \dots, N/2$ 上的 $X(k)$ 即可。将 $X(k)$ 写成极坐标形式

$$X(k) = |X(k)| e^{j \arg[X(k)]} \quad (3-62)$$

式中， $|X(k)|$ 称为幅频谱， $\arg[X(k)]$ 称为相频谱。





实际中也常常用信号的功率谱表示，功率谱是幅频谱的平方，功率谱PSD定义为

$$PSD(k) = \frac{|X(k)|^2}{N} \quad (3-63)$$

功率谱具有突出主频率的特性，在分析带有噪声干扰的信号时特别有用。将式（3-62）绘成的图形称为频谱图。由频谱图可以知道信号存在哪些频率分量，它们就是谱图中峰值对应的点。谱图中

最低频率为 $k=0$ ，对应实际频率为0（即直流）；

最高频率为 $k=N/2$ ，对应实际频率为 $f=f_s/2$ ；





对于处于 $0, 1, 2, \dots, N/2$ 上的任意点 k , 对应的实际频率为
 $f=kF=kf_s/N$ 。

由于所取单位不同, 频率轴有几种定标方式。图3-25列出频率轴几种定标方式的对应关系。

上图中 f 为归一化频率, 定义为

$$f' = \frac{f}{f_s} \quad (3-64)$$

f 无量纲, 在归一化频率谱图中, 最高频率为0.5。专用频谱分析仪器常用归一化频率表示。



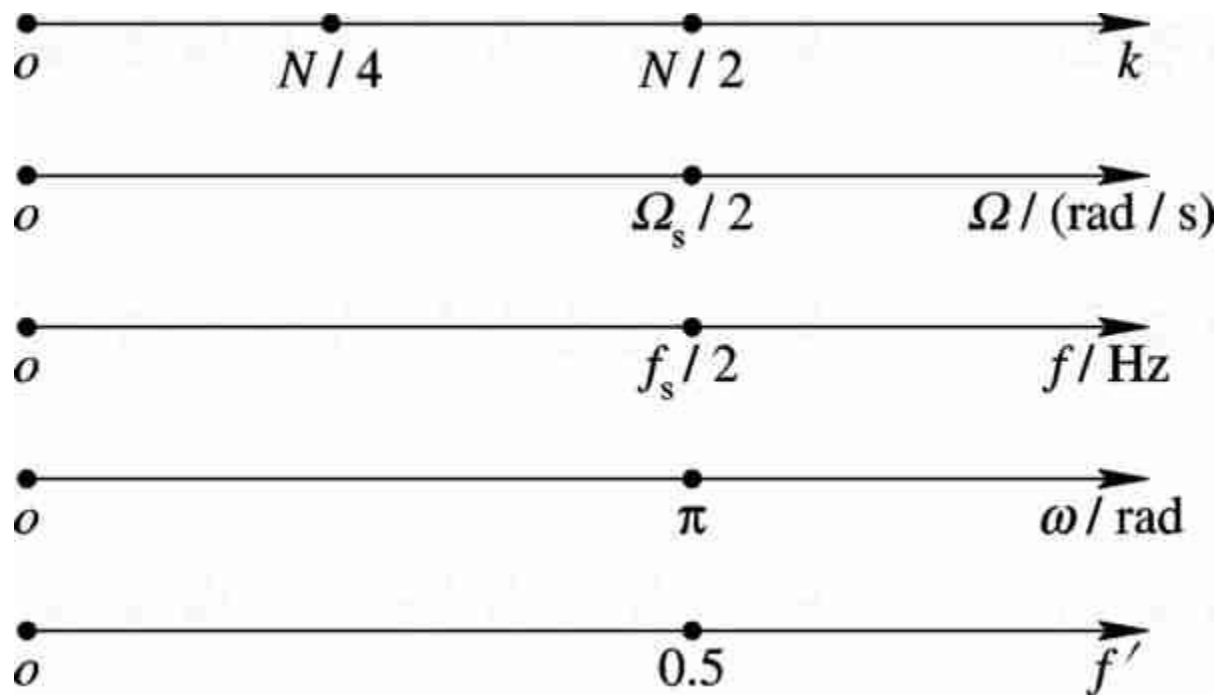


图3-25 模拟频率与数字频率之间的定标关系



〔例3-4〕 已知信号 $x(t)=0.15 \sin(2\pi f_1 t)+\sin(2\pi f_2 t)-0.1 \sin(2\pi f_3 t)$ ，其中， $f_1=1\text{Hz}$ ， $f_2=2\text{Hz}$ ， $f_3=3\text{Hz}$ 。 $x(t)$ 包含三个正弦波，但从时域波形图3-26 (a) 来看，似乎是一个正弦信号，很难看到小信号的存在，因为它被大信号所掩盖。取 $f_s=32\text{Hz}$ 作频谱分析。

解 因 $f_s=32\text{Hz}$ ，故

$$x(n) = x(nT) = 0.15 \sin\left(\frac{2\pi}{32} n\right) + \sin\left(\frac{4\pi}{32} n\right) - 0.1 \sin\left(\frac{6\pi}{32} n\right)$$





该信号为周期信号，其周期为 $N=32$ 。现对 $x(n)$ 作32点的离散傅里叶变换(DFT)，其幅度特性 $|X(k)|$ 如图3-26(b)所示。图中仅给出了 $k=0, 1, \dots, 15$ 的结果。 $k=16, 17, \dots, 32$ 的结果可由 $|X(N-k)|=|X(k)|$ 得出。因 $N=32$ ，故频率分辨率 $F=f_s/N=1\text{Hz}$ 。由图3-26(b)可知， $k=1, 2, 3$ 所对应的频谱即为频率 $f_1=1\text{ Hz}$ ， $f_2=2\text{ Hz}$ ， $f_3=3\text{Hz}$ 的正弦波所对应的频谱。幅频图如图3-26 (b)，该图中小信号成分清楚显示出来。可见小信号成分在时域中很难辨识而在频域中容易识别。



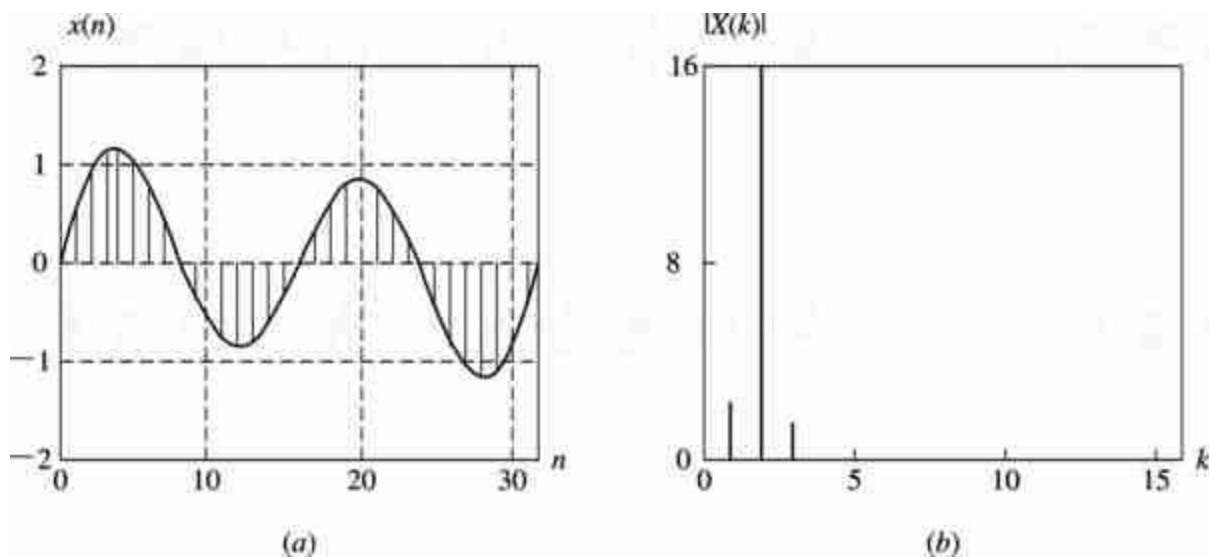


图 3-26 已知信号的时域图和幅频图



例 3-4 用 MATLAB 程序实现的过程如下：

```
clear all
f1=1;
f2=2;
f3=3;
fs=32;
N=32;
n=0:N-1;
xn=0.15*sin(2*pi*f1*n/fs)+sin(2*pi*f2*n/fs)-0.1*sin(
2*pi*f3*n/fs);
XK=fft(xn,N);           %计算序列 x(n) 的 N 点 FFT
magXK=abs(XK);          %计算幅频特性
```





```
phaXK=angle(XK);
```

%计算相频特性

```
subplot(1,2,1)
```

```
plot(n,xn)
```

%绘制时域信号图

```
grid on;
```

```
axis([0,31,-2,2]);
```

```
xlabel('n');ylabel('x(n)');
```

```
title('x(n) N=32');
```

```
subplot(1,2,2)
```

```
k=0:length(magXK)-1;
```

```
stem(k,magXK,'.');
```

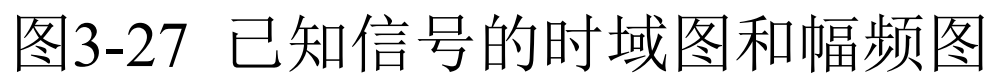
%绘制信号的幅频曲线图

```
axis([0,15,0,16]);
```

```
xlabel('k');ylabel('|X(k)|');
```

```
title('X(k)');
```







MATLAB中计算序列的离散傅里叶变换和逆变换是采用快速算法，利用fft和ifft函数实现。

函数fft用来求序列的DFT,调用格式为：

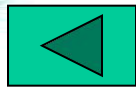
$$[XK]=fft(xn,N)$$

其中， xn 为有限长序列， N 为序列 xn 的长度， XK 为序列 xn 的DFT.

函数ifft用来求IDFT,调用格式为：

$$[xn]=ifft(XK,N)$$

其中， XK 为有限长序列， N 为序列 XK 的长度， xn 为序列 XK 的IDFT.





3.5 FFT的其他应用

3.5.1 线性卷积的FFT算法——快速卷积

1. FFT的快速卷积法

以FIR滤波器为例，因为它的输出等于有限长单位脉冲响应 $h(n)$ 与有限长输入信号 $x(n)$ 的离散线性卷积。

设 $x(n)$ 为 L 点， $h(n)$ 为 M 点，输出 $y(n)$ 为

$$y(n) = \sum_{m=0}^{M-1} h(m)x(n-m)$$





$y(n)$ 也是有限长序列，其点数为 $L+M-1$ 点。下面首先讨论直接计算线性卷积的运算量。由于每一个 $x(n)$ 的输入值都必须和全部的 $h(n)$ 值相乘一次，因而总共需要 LM 次乘法，这就是直接计算的乘法次数，以 m_d 表示为

$$m_d = LM \quad (3-65)$$

用FFT算法也就是用圆周卷积来代替这一线性卷积时，为了不产生混叠，其必要条件是使 $x(n)$ ， $h(n)$ 都补零值点，补到至少 $N=M+L-1$ ，即：

$$x(n) = \begin{cases} x(n) & 0 \leq n \leq L-1 \\ 0 & L \leq n \leq N-1 \end{cases} \quad h(n) = \begin{cases} h(n) & 0 \leq n \leq M-1 \\ 0 & M \leq n \leq N-1 \end{cases}$$



然后计算圆周卷积

$$y(n) = x(n) \textcircled{N} h(n)$$

这时， $y(n)$ 就能代表线性卷积的结果。

用FFT计算 $y(n)$ 的步骤如下：

(1) 求 $H(k)=\text{DFT} [h(n)]$ ， N 点；

(2) 求 $X(k)=\text{DFT} [x(n)]$ ， N 点；

(3) 计算 $Y(k)=X(k)H(k)$ ；

(4) 求 $y(n)=\text{IDFT} [Y(k)]$ ， N 点。

步骤(1)、(2)、(4)都可用FFT来完成。此时的工作量如下：

三次FFT运算共需要 $\frac{3}{2}N \lg N$ 次相乘，还有步骤③的N次相乘，
因此共需要相乘次数为

$$m_F = \frac{3}{2}N \lg N + N = N \left(1 + \frac{3}{2} \lg N \right) \quad (3-66)$$

比较直接计算线性卷积(简称直接法)和FFT法计算线性卷积(简称FFT法)这两种方法的乘法次数。设式(3-65)与式(3-66)

的比值为 K_m ，则

$$K_m = \frac{m_d}{m_F} = \frac{ML}{N \left(1 + \frac{3}{2} \lg N \right)}$$
$$= \frac{ML}{(M + L - 1) \left[1 + \frac{3}{2} \lg(M + L - 1) \right]} \quad (3-67)$$



分两种情况讨论如下：

(1) $x(n)$ 与 $h(n)$ 点数差不多。例如， $M=L$ ，则 $N=2M-1 \approx 2M$ ，则

$$K_m = \frac{M}{2\left(\frac{5}{2} + \frac{3}{2} \lg M\right)} = \frac{M}{5 + 3 \lg M}$$

这样可得下表：

$M=L$	8	16	32	64	128	256	512	1024	2048	4096
K_m	0.572	0.941	1.6	2.78	5.92	8.82	16	29.24	53.9	99.9

当 $M=8$ 时，FFT法的运算量大于直接法；当 $M=32$ 时，二者相当；当 $M=512$ 时，FFT法运算速度可快16倍；当 $M=4096$ 时，FFT法约快100倍。可以看出，当 $M=L$ 且 M 超过32以后， M 越长，FFT法的好处越明显。因而将**圆周卷积称为快速卷积**。



(2) 当 $x(n)$ 的点数很多时, 即当 $L \gg M$ 。通常不允许等 $x(n)$ 全部采集齐后再进行卷积; 否则, 使输出相对于输入有较长的延时。此外, 若 $N=L+M-1$ 太大, $h(n)$ 必须补很多个零值点, 很不经济, 且FFT的计算时间也要很长。这时FFT法的优点就表现出来了, 因此需要采用分段卷积或称分段过滤的办法。即将 $x(n)$ 分成点数和 $h(n)$ 相仿的段, 分别求出每段的卷积结果, 然后用一定方式把它们合在一起, 便得到总的输出, 其中每一段的卷积均采用FFT方法处理。有两种分段卷积的办法: 重叠相加法和重叠保留法。





2. 重叠相加法

设 $h(n)$ 的点数为 M ，信号 $x(n)$ 为很长的序列。我们将 $x(n)$ 分解为很多段，每段为 L 点， L 选择成和 M 的数量级相同，用 $x_i(n)$ 表示 $x(n)$ 的第 i 段：

$$x_i(n) = \begin{cases} x(n) & iL \leq n \leq (i+1)L-1 \\ 0 & \text{其他}n \end{cases} \quad i=0, 1, \dots \quad (3-68)$$

则输入序列可表示成

$$x(n) = \sum_{i=0}^{\infty} x_i(n) \quad (3-69)$$





这样， $x(n)$ 和 $h(n)$ 的线性卷积等于各 $x_i(n)$ 与 $h(n)$ 的线性卷积之和，即

$$y(n) = x(n) * h(n) = \sum_{i=0}^{\infty} x_i(n) * h(n) \quad (3-70)$$

每一个 $x_i(n)*h(n)$ 都可用上面讨论的快速卷积办法来运算。由于 $x_i(n)*h(n)$ 为 $L+M-1$ 点，故先对 $x_i(n)$ 及 $h(n)$ 补零值点，补到 N 点。为便于利用基-2 FFT算法，一般取 $N=2^m \geq L+M-1$ ，然后作 N 点的圆周卷积：

$$y_i(n) = x_i(n) \textcircled{N} h(n)$$

由于 $x_i(n)$ 为 L 点，而 $y_i(n)$ 为 $(L+M-1)$ 点（设 $N=L+M-1$ ），故相邻两段输出序列必然有 $(M-1)$ 个点发生重叠，即前一段的后 $(M-1)$ 个点和后一段的前 $(M-1)$ 个点相重叠，如图3-27所示。按照式（3-70），应该将重叠部分相加再和不重叠的部分共同组成输出 $y(n)$ 。



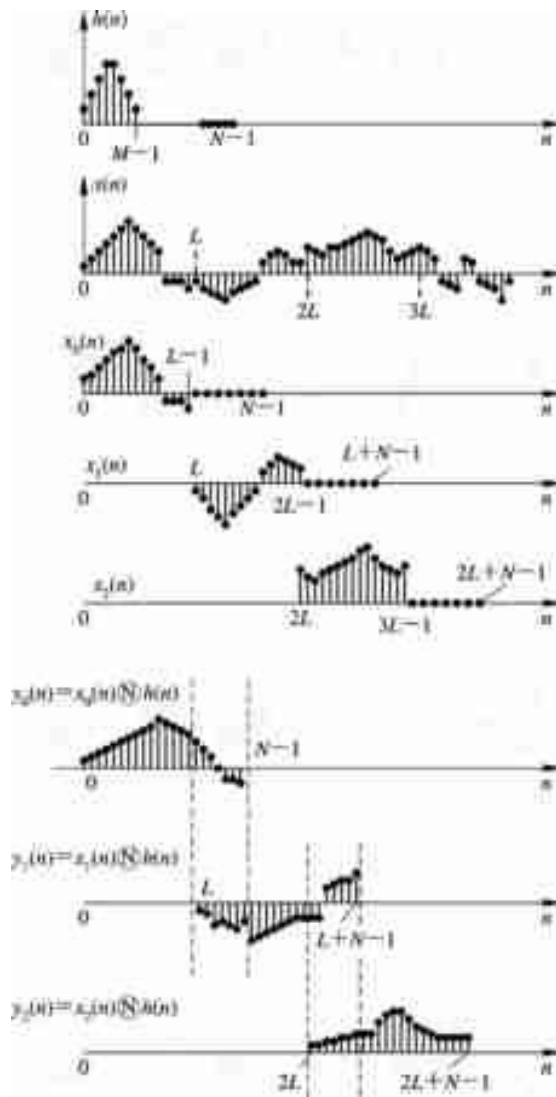


图 3-28 重叠相加法图形



和上面的讨论一样，用 FFT 法实现重叠相加法的步骤如下：

(1) 计算 N 点FFT， $H(k)=\text{DFT} [h(n)]$ ；

(2) 计算 N 点FFT， $X_i(k)=\text{DFT} [x_i(n)]$ ；

(3) 相乘， $Y_i(k)=X_i(k)H(k)$ ；

(4) 计算 N 点IFFT， $y_i(n)=\text{IDFT} [Y_i(k)]$ ；

(5) 将各段 $y_i(n)$ （包括重叠部分）相加，
$$y(n) = \sum_{i=0}^{\infty} y_i(n)$$

重叠相加的名称是由于各输出段的重叠部分相加而得名的。





3. 重叠保留法

此方法与上述方法稍有不同。先将 $x(n)$ 分段，每段 $L=N-M+1$ 个点，这是相同的。不同之处是，序列中补零处不补零，而在每一段的前边补上前一段保留下来的 $(M-1)$ 个输入序列值，组成 $L+M-1$ 点序列 $x_i(n)$ ，如图3-29(a)所示。如果 $L+M-1 < 2^m$ ，则可在每段序列末端补零值点，补到长度为 2^m ，这时如果用DFT实现 $h(n)$ 和 $x_i(n)$ 圆周卷积，则其每段圆周卷积结果的前 $(M-1)$ 个点的值不等于线性卷积值，必须舍去。



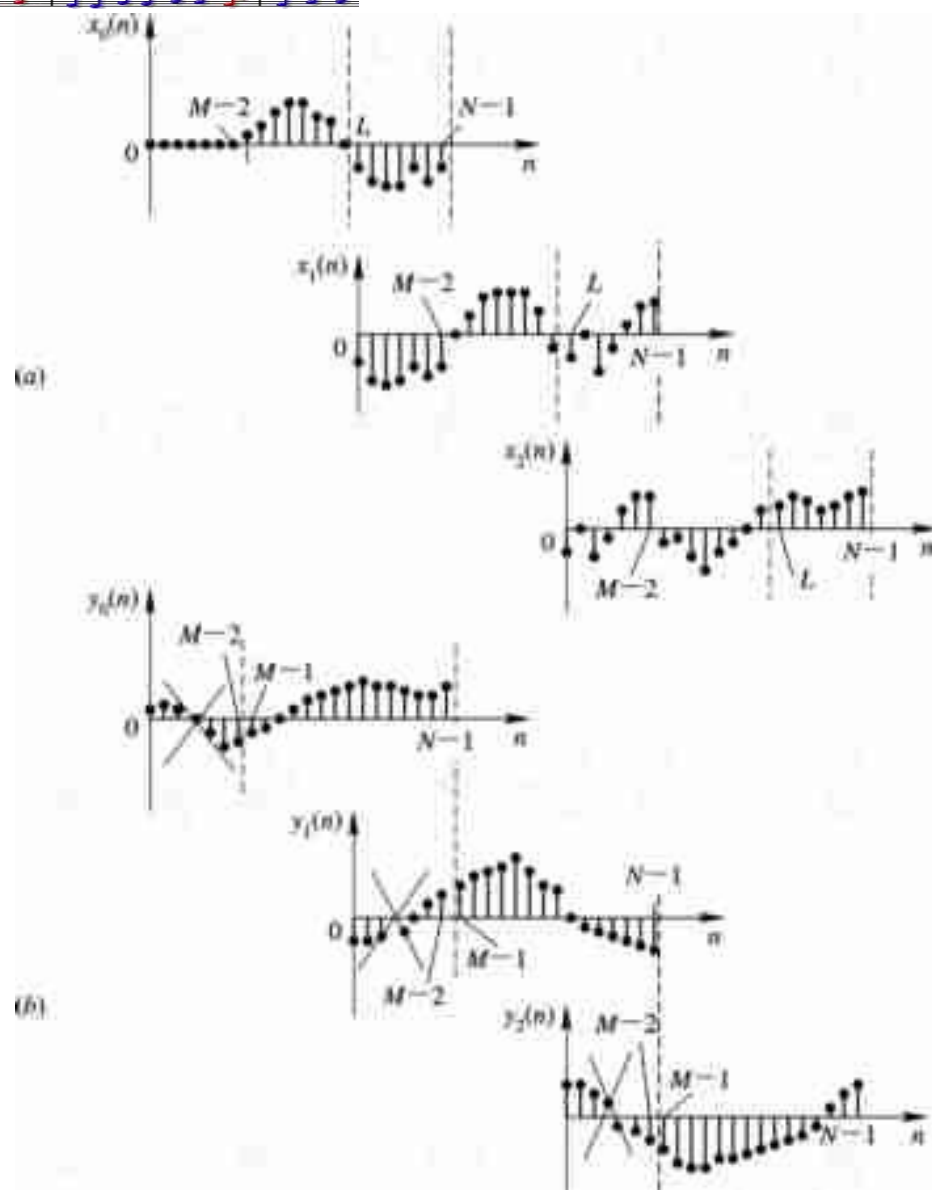


图 3-29 重叠保留法示意图



为了说明以上说法的正确性，我们来看一看图3-29。任一段 $x_i(n)$ （为 N 点）与 $h(n)$ （原为 M 点，补零值后也为 N 点）的 N 点圆周卷积

$$y_i(n) = x_i(n) \textcircled{N} h(n) = \sum_{m=0}^{N-1} x_i(m) h((n-m))_N R_N(n) \quad (3-71)$$

由于 $h(m)$ 为 M 点，补零后作 N 点圆周移位时，在 $n=0,1,\dots,M-2$ 的每一种情况下， $h((n-m))_N R_N(m)$ 在 $0 \leq m \leq N-1$ 范围的末端出现非零值，而此处 $x_i(m)$ 是有数值存在的，图3-29（c），（d）为 $n=0, n=M-2$ 的情况，所以在

$$0 \leq n \leq M-2$$





这一部分的 $y_i(n)$ 值中将混入 $x_i(m)$ 尾部与 $h((n-m))_N \cdot R_N(m)$ 尾部的乘积值，从而使这些点的 $y_i(n)$ 不同于线性卷积结果。但是从 $n=M-1$ 开始到 $n=N-1$ ， $h((n-m))_N R_N(m) = \mathbf{h}(n-m)$ （如图3-29（e），（f）所示），圆周卷积值完全与线性卷积值一样， $y_i(n)$ 就是正确的线性卷积值。因而必须把每一段圆周卷积结果的前 $(M-1)$ 个值去掉，如图3-29（g）所示。

因此，为了不造成输出信号的遗漏，对输入分段时，就需要使相邻两段有 $M-1$ 个点重叠（对于第一段，即 $x_0(n)$ ，由于没有前一段保留信号，则需要序列前填充 $M-1$ 个零值点），这样，若原输入序列为 $x'(n)$ （ $n \geq 0$ 时有值），则应重新定义输入序列





$$x(n) = \begin{cases} 0 & 0 \leq n \leq M-2 \\ x'[n - (M - 1)] & M-1 \leq n \end{cases} \quad (3-72a)$$

而

$$x_i(n) = \begin{cases} x[n + i(N - M + 1)] & 0 \leq n \leq N-1 \\ 0 & \text{其他 } n \end{cases}$$
$$i=0, 1, \dots \quad (3-72b)$$





在这一公式中，已经把每一段的时间原点放在该段的起始点，而不是 $x(n)$ 的原点。这种分段方法示于图3-28中，每段 $x_i(n)$ 和 $h(n)$ 的圆周卷积结果以 $y_i(n)$ 表示，如图3-28(b)所示，图中已标出每一段输出段开始的 $(M-1)$ 个点， $0 \leq n \leq M-2$ 部分舍掉不用。把相邻各输出段留下的序列衔接起来，就构成了最后的正确输出，即

$$y(n) = \sum_{i=0}^{\infty} y'_i[n - i(N - M + 1)] \quad (3-73)$$

式中：

$$y'_i(n) = \begin{cases} y_i(n) & M-1 \leq n \leq N-1 \\ 0 & \text{其他}n \end{cases} \quad (3-74)$$

这时，每段输出的时间原点放在 $y'_i(n)$ 的起始点，而不是 $y(n)$ 的原点。



3.5.2 信号消噪

假设信号在传输过程中，受到噪声的干扰，则在接收端得到的信号由于受到噪声的干扰，信号将难以辨识。用FFT方法消噪就是对含噪信号的频谱进行处理，将噪声所在频段的 $X(k)$ 值全部置零后，再对处理后的 $X(k)$ 进行离散傅里叶反变换（IFFT）可得原信号的近似结果。这种方法要求噪声与信号的频谱不在同一频段，否则，将很难处理。





例3-5 将上述消噪原理用于语音消噪。图3-30是一个实际例子，语音信号受到了强烈的啸叫噪声干扰，无法听清语音意思，如图3-30(a)，信号淹没在噪声中（信噪比只有-10dB）。试用FFT方法消噪。

先作FFT分析，得到其功率谱如图3-30(b)，可见在频率2.5 kHz附近有一极强分量。这就是啸叫噪声干扰。图3-30(b)中频率在30~800Hz范围是语音信号。对频谱进行修正，去除噪声频段，即将大于2.5 kHz部分的 $X(k)$ 值全部置为零，图3-30(c)是去噪后的功率谱。再由反变换（IFFT）重构信号得到原语音信号如图3-30(d)。这时信噪比为14dB，提高了24dB。这就是早期的数字式录音音乐中所采用的消噪方法。



第3章 快速傅里叶变换

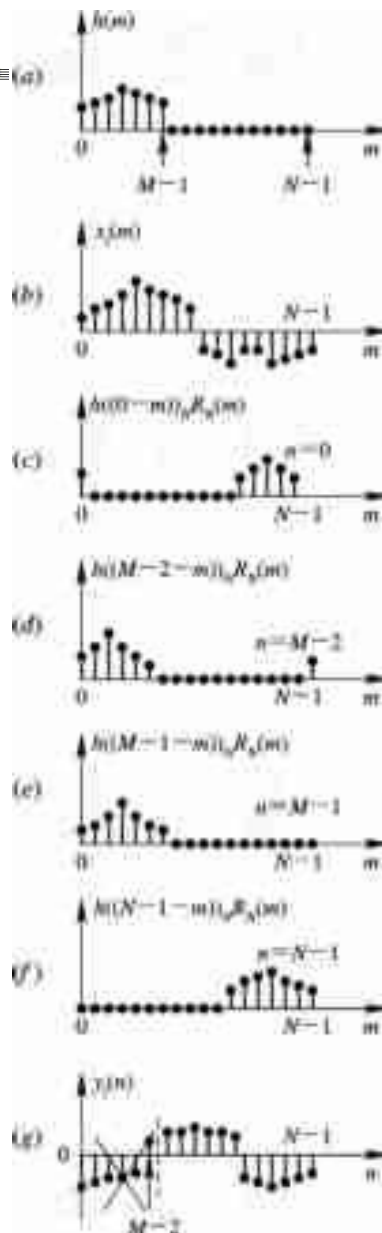


图3-30 用保留信号代表补零后的局部混叠现象

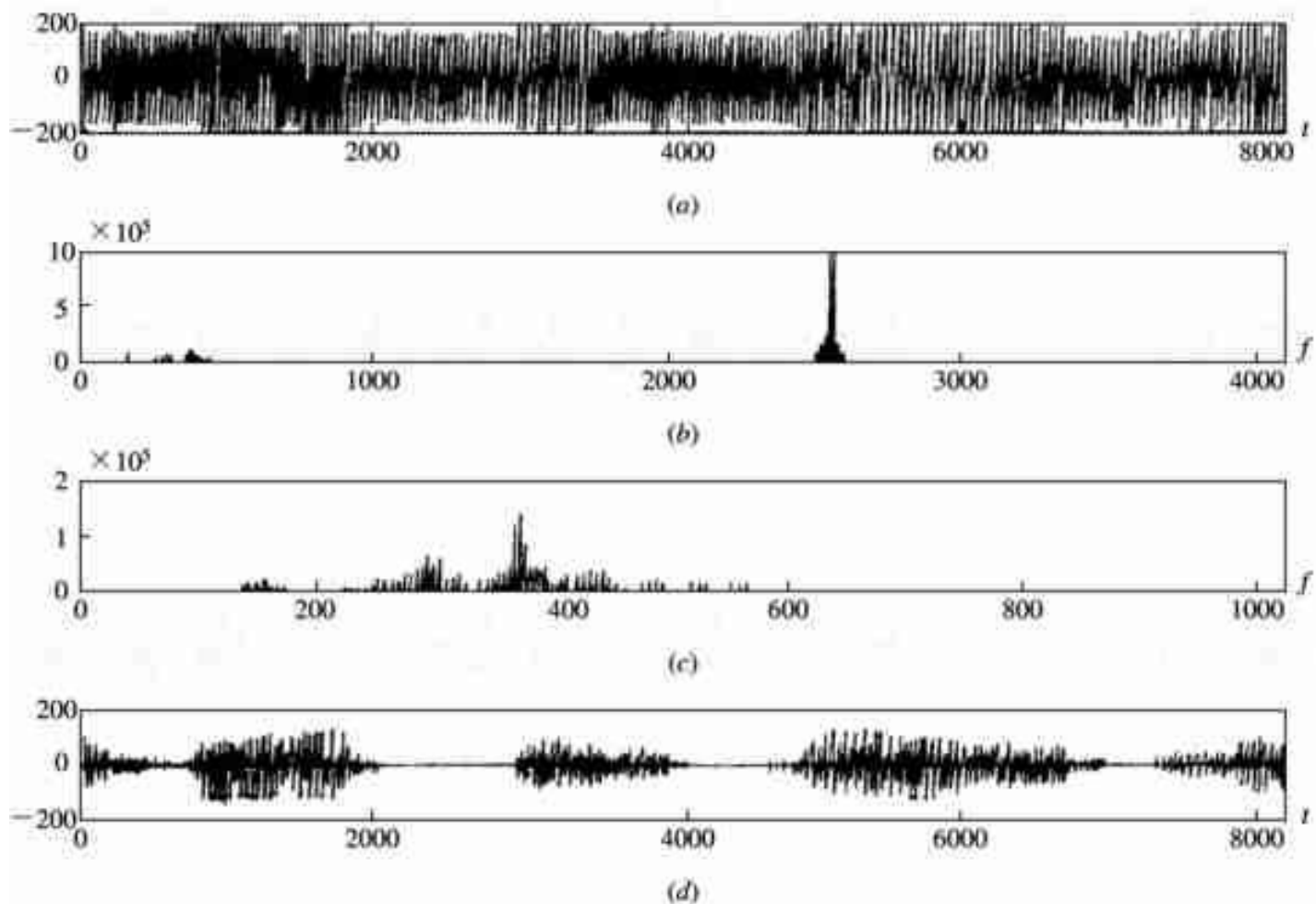


图 3-31 语音信号消噪过程

(a) 信号淹没在啸叫噪声中； (b) 信号与噪声的功率谱；
(c) 去噪后的功率谱； (d) 重构原语音信号